

Pontifícia Universidade Católica de Minas Gerais

Disciplina: Testes de Software

Trabalho: test_pattern

Aluno: Pedro Luis Gonçalves

Matrícula: 729167

Builders

No teste deste projeto, o CarrinhoBuilder foi preferido em vez de um hipotético CarrinhoMother porque o carrinho é um objeto mais flexível e sujeito a variações entre cenários. Em alguns testes ele precisa conter itens, em outros precisa estar vazio, em outros precisa trocar o usuário associado e, às vezes, precisa mudar apenas uma parte desses dados. Se usássemos um Object Mother para o carrinho, provavelmente surgiriam muitos métodos específicos para cada combinação possível, como carrinho premium com dois itens, carrinho vazio ou carrinho com desconto, o que deixaria a API inchada e pouco escalável.

O CarrinhoBuilder resolve isso com uma API fluente e explícita. O construtor já entrega um estado padrão útil, com um usuário comum e um item padrão, e os métodos comUser(user), comItens([...]) e vazio() permitem alterar apenas o que é relevante para cada teste. Isso melhora a legibilidade porque o teste passa a mostrar a intenção do cenário, e não os detalhes mecânicos da montagem do objeto.

Antes, o setup era mais verboso e manual, exigindo a criação direta do usuário, do item e do carrinho em cada teste. Depois, com o Builder, o cenário fica centralizado e mais claro, pois basta descrever o que muda em relação ao padrão. Na prática, isso reduz o ruído do teste e facilita a manutenção. Se o formato interno do Carrinho mudar, a maioria dos testes continuará estável, pois a criação do objeto fica centralizada em um único ponto.

Mocks vs Stubs

No teste de sucesso do cliente Premium, o GatewayPagamento foi usado como Stub e o EmailService como Mock. Essa distinção é importante porque cada dependência tem um papel diferente no teste.

O GatewayPagamento foi tratado como Stub porque o objetivo dele era apenas fornecer um retorno controlado para direcionar o fluxo do CheckoutService. No cenário Premium, ele retorna success true, permitindo que o processo continue até o final. O foco aqui é a verificação de estado: queremos garantir que o serviço calcula corretamente o total com desconto de 10% e prossegue com a criação do pedido. O teste não depende de observar chamadas internas do gateway além do necessário para confirmar o valor cobrado.

Já o EmailService foi usado como Mock porque o comportamento esperado é a interação em si. Neste caso, não basta apenas deixar o fluxo continuar; é preciso verificar se o e-mail foi enviado uma vez, para o destinatário correto, com o assunto correto. Aqui o foco é verificação de comportamento: o teste confirma que o CheckoutService executou a integração com a notificação conforme esperado.

Em resumo, o GatewayPagamento funciona como Stub porque controla o retorno e sustenta a verificação de estado. O EmailService funciona como Mock porque permite verificar a interação e o comportamento esperado.

Conclusão

O uso deliberado de Padrões de Teste neste trabalho ajuda a evitar Test Smells comuns, como setup obscuro, excesso de duplicação e testes acoplados demais à montagem dos objetos. O UserMother simplifica a criação de usuários comuns e reutilizáveis. O CarrinhoBuilder organiza cenários mais complexos sem espalhar detalhes de construção pelo teste. Já os doubles, usados como Stubs e Mocks, deixam claro o que está sendo controlado e o que está sendo verificado.

Com isso, a suíte de testes fica mais expressiva, mais fácil de manter e mais resistente a mudanças. Em vez de testes frágeis e difíceis de entender, temos testes que comunicam intenção e protegem a regra de negócio de forma sustentável.